a) 5 b) 6 c) 7 d) Error

Q39. What does r"\n" contain?

a) A newline character b) Two chars: backslash + n c) An empty string d) Error

Q40. What does "Python".startswith(("py","Py")) return?

a) Error - tuple not allowed b) True c) False d) [False, True]

Q41. What does "abcabc".replace("b","B") return?

a) "aBcabc" b) "aBcaBc" c) "abcaBc" d) "aBc"

Q42. What does re.findall(r"\d+", "a1b22c333") return?

a) ["1","22","333"] b) "1","22","333" (separate values) c) ["1","2","2","3","3","3"] d) ["1","2","3"]

Q43. What does re.search(r"\bword\b", "keyword") return?

a) A match object b) None c) "word" d) Error

Q44. What does "hello"[10] raise?

a) None b) "" c) IndexError d) " "

Q45. What does "a:b:c:d".split(":", 2) return?

a) ["a","b","c","d"] b) ["a","b","c:d"] c) ["a:b:c:d"] d) ["a","b:c:d"]

Q46. Which regex pattern matches ONE OR MORE digits?

a) \d b) \d+ c) \d* d) \D+

Q47. What does " ".strip() return?

a) " " b) "" c) None d) " "

Q48. What does re.sub(r"[aeiou]", "*", "hello") return?

a) "h*llo" b) "h*ll*" c) "h***o" d) "*ello"

Q49. What does "abcdef"[::2] return?

a) "ace" b) "bdf" c) "abc" d) "def"

Q50. When should you use re.compile() over calling re.search() directly?

a) Always - compile is always faster b) When the same pattern is used many times in a loop c) Never - they are identical d) Only when using capture groups

TOPIC 3: LISTS, TUPLES & COMPREHENSIONS (Q51–Q75)

Q51. What does the following print?
a = [1,2,3]; b = a; b.append(4); print(a)

a) [1,2,3] b) [1,2,3,4] c) Error d) [1,2,3,[4]]

Q52. What does a=[1,2,3]; b=a[:] create for b?

a) An alias for a b) A shallow copy of a c) A deep copy of a d) None

Q53. What does [1,2,3].pop() return?

a) 1 b) [1,2] c) 3 d) [1,2,3]

Q54. What does sorted([3,1,4,1,5,9], reverse=True) return?

a) [9,5,4,3,1,1] b) [1,1,3,4,5,9] c) [9,5,4,3,1,1] with original unchanged d) Both A and C

Q55. What does [x**2 for x in range(5) if x%2==0] produce?

a) [0,4,16] b) [1,9,25] c) [0,1,4,9,16] d) [4,16]

Q56. What is len((1,))?

a) Error (needs brackets) b) 0 c) 1 d) 2

Q57. Why prefer tuple over list for fixed, hashable data?

a) Tuples are mutable b) Tuples can be used as dict keys; lists cannot c) Tuples support more methods d) Tuples are slower but safer

Q58. What does list(range(1,10,3)) produce?

a) [1,3,6,9] b) [1,4,7] c) [1,4,7,10] d) [3,6,9]

Q59. What does the following print?
data=[1,[2,3],4]; data[1].append(5); print(data)

a) [1,[2,3,5],4] b) [1,[2,3,4,5]] c) Error d) [1,[2,3],4]

Q60. What does [i*j for i in range(3) for j in range(3)] produce?

a) [[0,0,0],[0,1,2],[0,2,4]] b) [0,0,0,0,1,2,0,2,4] c) [(0,0),(0,1),...] tuples d) Error

Q61. What does tuple([1,2,3]) + (4,5) return?

a) [1,2,3,4,5] b) (1,2,3,4,5) c) (1,2,3,[4,5]) d) Error

Q62. What does max(["banana","apple","cherry"]) return?

a) "cherry" b) "banana" c) Error d) "apple"

Q63. When you do a, b, *c = [1,2,3,4,5], what is c?

a) [3] b) [4,5] c) [3,4,5] d) (3,4,5)

Q64. What does list(zip([1,2,3],[4,5])) return?

a) [(1,4),(2,5),(3,None)] b) [(1,4),(2,5),(3)] c) [(1,4),(2,5)] d) Error

Q65. What does sorted([(1,"b"),(2,"a"),(1,"a")], key=lambda x:(x[0],x[1])) return?

a) [(1,"a"),(1,"b"),(2,"a")] b) [(1,"b"),(1,"a"),(2,"a")] c) [(2,"a"),(1,"a"),(1,"b")] d) Error

Q66. What does list(enumerate(["a","b","c"], start=1)) return?

a) [(0,"a"),(1,"b"),(2,"c")] b) [(1,"a"),(2,"b"),(3,"c")] c) [1,2,3] d) ["a","b","c"]

Q67. What is the key difference between list.sort() and sorted(list)?

a) No difference b) sort() modifies in place, returns None; sorted() returns new list c) sorted() modifies in place; sort() returns new list d) sort() only works on numbers

Q68. What does [x for x in range(10) if x%3==0 if x%2==0] return?

a) [0,6] b) [0,3,6,9] c) [0,2,4,6,8] d) [6]

Q69. What does [1,2,3].index(5) raise?

a) None b) -1 c) ValueError d) IndexError

Q70. Which is more memory-efficient for large sequences?

a) List comprehension b) Generator expression c) Both use same memory d) Depends on element type

Q71. What does any([0, None, False, [], ""]) return?

a) True b) False c) None d) []

Q72. What is all([1, "hello", 3.14, True])?

a) False b) True c) 1 d) Error

Q73. What does [0]*3 produce?

a) [0,0,0] b) [0,3] c) 0 d) Error

Q74. What does list(reversed([1,2,3])) return?

a) [3,2,1] b) [1,2,3] c) a reversed object d) Error

Q75. When is a regular for loop preferable to a list comprehension?

a) Never - comprehensions always win b) When the body has multiple operations or side effects c) When data has >100 elements d) When using if conditions

TOPIC 4: DICTIONARIES & SETS (Q76–Q100)

Q76. What does d={"a":1}; d.get("b", 0) return?

a) None b) KeyError c) 0 d) {}

Q77. What happens after d={"x":1,"y":2}; d.update({"y":10,"z":3})?

		with any of them.
41	**B**	Without a count limit, replace() replaces ALL occurrences.
42	**A**	\d+ matches one or more consecutive digits as a group. findall returns all non-overlapping matches.
43	**B**	\b is a word boundary. In "keyword", "word" is inside another word so boundaries don't match.
44	**C**	Strings use index-based access. Index 10 on a 5-char string raises IndexError.
45	**B**	maxsplit=2 means split at most twice, producing 3 parts. The remaining string stays unsplit.
46	**B**	\d matches exactly one digit. + means "one or more". * means "zero or more". \D matches non-digits.
47	**B**	strip() removes all whitespace. A string of only whitespace becomes an empty string.
48	**B**	[aeiou] matches any vowel. In "hello": e and o are vowels → h*ll*. h and l are consonants.
49	**A**	[::2] takes every other character starting from index 0: a(0), c(2), e(4) = "ace".
50	**B**	re.compile() pre-compiles the pattern into a regex object, avoiding recompilation on every loop iteration.
51	**B**	b = a creates an alias - both names point to the same list object. Appending via b also changes a.
52	**B**	Slicing creates a shallow copy. For nested lists, nested objects are still shared.
53	**C**	pop() with no argument removes and returns the LAST element. pop(0)

		removes the first.
54	**D**	sorted() always returns a NEW list without modifying the original. The result is in descending order.
55	**A**	range(5)=[0,1,2,3,4]. Evens: 0,2,4. Squared: 0,4,16.
56	**C**	(1,) is a 1-element tuple. The trailing comma is required - (1) is just parenthesised int 1.
57	**B**	Tuples are hashable (if their elements are), so they can be dict keys or set members. Lists cannot.
58	**B**	Starting at 1, stepping by 3: 1, 4, 7, 10(>=10 stop). Result: [1,4,7].
59	**A**	data[1] is the nested list [2,3]. Appending to it modifies it in place. Shallow copy shares nested refs.
60	**B**	Nested comprehensions flatten to one list: 0*0,0*1,0*2,1*0,1*1,1*2,2*0,2*1,2*2.
61	**B**	tuple + tuple = concatenated tuple. Both sides must be tuples.
62	**A**	String comparison is lexicographic. c > b > a, so "cherry" is the max.
63	**C**	*c captures all remaining elements after a=1 and b=2: [3,4,5] as a list.
64	**C**	zip() stops at the shortest iterable. Only 2 pairs are produced; element 3 is dropped.
65	**A**	Sort by first element then second. (1,"a") < (1,"b") < (2,"a").
66	**B**	start=1 makes counting start from 1 instead of 0.
67	**B**	This is a classic interview question. list.sort() → None. sorted() → new list, original unchanged.

68	**A**	Multiple if clauses are AND'd. Divisible by both 3 and 2 = divisible by 6: 0, 6.
69	**C**	index() raises ValueError when the value is not found. Use "5 in lst" to check first.
70	**B**	Generator expressions are lazy - they yield one item at a time without storing all in memory.
71	**B**	any() returns True if AT LEAST ONE element is truthy. All elements here are falsy.
72	**B**	all() returns True if EVERY element is truthy. 1, non-empty strings, non-zero floats, and True are all truthy.
73	**A**	List multiplication repeats the list. [0]*3 = [0,0,0]. Warning: for mutable objects like lists, use list comprehension instead.
74	**A**	reversed() returns an iterator. list() materialises it: [3,2,1].
75	**B**	Comprehensions are for building lists. If you're printing, logging, or updating multiple variables, use a for loop for clarity.
76	**C**	get() returns the default (0 here) when the key is absent. No exception is raised.
77	**B**	update() merges dicts, overwriting existing keys with new values and adding new keys.
78	**C**	Accessing a missing dict key raises KeyError. Use .get() or check with "key in d" first.
79	**C**	Dict keys must be hashable. Tuples (with hashable elements) are hashable. Lists, dicts, and sets are not.